

StudyPlanner - Dokumentacja projektu Programowanie Aplikacji Webowych

StudyPlanner

Aplikacja do zarządzania sesjami nauki z inteligentną analizą przerw

Przedmiot: Programowanie Aplikacji Webowych

Technologia: React + Express + Prisma + SQLite + TypeScript

Zespół: 1 osoba

Termin: 31 Maja 2026

1. Opis aplikacji

1.1 Cel i przeznaczenie

StudyPlanner to nowoczesna aplikacja webowa przeznaczona do osobistego zarządzania czasem, planowania bloków edukacyjnych oraz dynamicznej optymalizacji przerw w celu zachowania najwyższej wydajności umysłowej [cite: 42, 43]. System umożliwia wygodne dodawanie realizowanych sesji nauki, definiowanie ich parametrów, przeglądanie historii zaplanowanych aktywności oraz automatyczną analizę obciążenia kognitywnego[cite: 43].

Głównym celem aplikacji jest pomoc użytkownikom w efektywnej organizacji pracy i przeciwdziałaniu zmęczeniu umysłowemu — dzięki inteligentnemu algorytmowi system automatycznie wylicza sugerowany, optymalny czas odpoczynku po zakończeniu każdego bloku, dostosowany dynamicznie do zadeklarowanej intensywności i czasu trwania zadania[cite: 44].

1.2 Funkcjonalności

Zarządzanie sesjami nauki

- Dodawanie nowej sesji naukowej z nazwą tematu, czasem trwania wyrażonym w minutach oraz poziomem trudności [cite: 47]
- Dynamiczne wyświetlanie pełnej listy zaplanowanych aktywności bez konieczności przeładowywania strony (SPA) [cite: 43]
- Precyzyjne rejestrowanie czasu utworzenia każdego bloku nauki (DateTime) bezpośrednio w strukturze bazy danych [cite: 47]

Inteligentny system rekomendacji przerw

- Automatyczna analiza intensywności wysiłku kognitywnego na podstawie wprowadzonych danych po stronie backendu [cite: 44]
- Kalkulacja sugerowanego czasu odpoczynku bazująca na algorytmie matematycznym (modyfikacja bazy 20% czasu sesji)
- Generowanie i wstrzykiwanie spersonalizowanych komunikatów z rekomendacjami bezpośrednio na kafelkach zadań użytkownika

Interfejs i UX

- Czytelny, nowoczesny kafelkowy układ prezentacji danych (Grid Dashboard)
- Pełna responsywność interfejsu (RWD) idealnie dopasowana do ekranów komputerów desktopowych oraz urządzeń mobilnych [cite: 23]
- Zastosowanie asynchronicznej komunikacji sieciowej JSON zapewniającej wysoką płynność i responsywność interfejsu [cite: 17, 22]

Ustawienia i architektura

- Ścisłe reguły walidacji danych wejściowych w polach formularza planowania
- Pełne uniezależnienie i bezpieczna separacja warstwy klienckiej od serwerowej (CORS) [cite: 14]

1.3 Moduły i komponenty aplikacji

Aplikacja została zrealizowana w architekturze Single Page Application (SPA), a jej interfejs składa się z następujących zintegrowanych modułów operacyjnych (spełniając wymogi techniczne specyfikacji)[cite: 14, 178]:

Ekran / Moduł | Nazwa komponentu | Opis

1 | Główny Kontener (App.tsx) | Zarządzanie stanem danych (state), koordynacja przepływu informacji oraz asynchroniczna synchronizacja z API [cite: 179]

2 | Formularz Planowania (StudyForm.tsx) | Panel wejściowy użytkownika; zawiera walidację pól, obsługę wyboru stopnia trudności oraz wysyłanie żądań POST [cite: 182]

3 | Dashboard / Lista Sesji (StudyList.tsx) | Prezentacja zaplanowanych bloków w układzie siatki z wbudowanym modułem inteligentnych powiadomień [cite: 182]

1.4 Wymagania techniczne — spełnienie

Wymaganie | Status | Realizacja

Rozdzielony frontend i backend | ✓ Spełnione | Kompletna separacja warstwy klienckiej (port 3000) i serwerowej (port 5000) [cite: 14]

Nowoczesny serwer webowy | ✓ Spełnione | Stabilne środowisko uruchomieniowe REST API oparte na Node.js i Express.js [cite: 15, 17]

Integracja z bazą danych | ✓ Spełnione | Zastosowanie lokalnej bazy SQLite obsługiwanej przez nowoczesne narzędzie Prisma ORM [cite: 16]

Warstwa kliencka (SPA) | ✓ Spełnione | Zbudowanie wydajnej architektury komponentów interfejsu w bibliotece React.js [cite: 20]

Użycie silnego typowania TypeScript | ✓ Spełnione | Cały kod źródłowy projektu (zarówno klient, jak i serwer) posiada ścisłe reguły typowania danych [cite: 36]

Stylowanie Desktop i Mobile | ✓ Spełnione | Implementacja mechanizmów CSS Grid oraz Flexbox z regułami Media Queries (RWD) [cite: 23]

Pokrycie kodu testami jednostkowymi | ✓ Spełnione | Zestaw testów automatycznych logiki biznesowej oraz komponentów w frameworku Jest [cite: 18, 24]

2. Opis podziału pracy

Projekt realizowany jest w całości jednoosobowo przez technika programistę[cite: 189].
Poniższa tabela przedstawia szczegółowy podział przeprowadzonych prac projektowych na poszczególne etapy wraz z faktycznym nakładem czasowym[cite: 189]:

Etap | Zakres prac | Czas (h)

1. Dokumentacja | Analiza wymagań technicznych, opis systemu, schemat bazy i mapy przepływu danych [cite: 190] | 3h
 2. Projekt i konfiguracja | Inicjalizacja repozytorium Git, konfiguracja środowisk tsconfig, setup Vite i serwera Express [cite: 190] | 4h
 3. Warstwa danych | Przygotowanie pliku schema.prisma, generowanie klienta bazodanowego, utworzenie bazy SQLite [cite: 190] | 5h
 4. Logika API i Kontrolery | Oprogramowanie tras HTTP (routes), implementacja endpointów REST API, konfiguracja CORS [cite: 190] | 6h
 5. Testy Jednostkowe backendu | Implementacja testów automatycznych w Jest weryfikujących zachowanie algorytmu przerw [cite: 190] | 4h
 6. Architektura UI (Frontend) | Konfiguracja stanu useState, oprogramowanie hooków efektów ubocznych useEffect [cite: 190] | 5h
 7. Komponenty formularza | Implementacja komponentu StudyForm, obsługa zdarzeń formularza, walidacja pól wejściowych [cite: 190] | 6h
 8. Komponenty listy i powiadomień | Implementacja komponentu StudyList, renderowanie kafelków wraz z rekomendacjami przerw [cite: 190] | 5h
 9. Stylowanie responsywne | Wdrożenie reguł CSS dopasowujących interfejs do widoków desktopowych oraz mobilnych [cite: 190] | 4h
 10. Testy frontendu i optymalizacja | Testy renderowania komponentów w środowisku izolowanym Jest, przygotowanie paczki ZIP [cite: 190] | 3h
- ŁĄCZNIE | | 45h

3. Baza danych

3.1 Opis struktury

Zgodnie z załączonymi wymaganiami lekkości, niezależności oraz łatwości uruchomienia aplikacji, system korzysta z lokalnej relacyjnej bazy danych SQLite[cite: 16, 194]. Zarządzanie bazą, migracjami struktury oraz operacjami odczytu/zapisu odbywa się za pomocą mapowania obiektowo-relacyjnego (ORM) Prisma[cite: 16]. Baza opiera się na znormalizowanej tabeli zasilającej bezpośrednio interfejs[cite: 195].

3.2 Opis tabeli systemowej

Tabela: StudySession

Przechowuje pełne informacje na temat wszystkich zaplanowanych i zrealizowanych bloków nauki użytkownika[cite: 198].

Kolumna | Typ (Prisma) | Klucz | Opis

id | String | PK | Unikalny identyfikator sesji generowany automatycznie jako ciąg znaków UUID [cite: 199]

title | String | NOT NULL | Temat lub specyficzna nazwa zagadnienia edukacyjnego (np. "Fizyka Jądrowa") [cite: 199]

duration | Int | NOT NULL | Czas trwania bloku nauki wyrażony w minutach (liczba całkowita dodatnia) [cite: 199]

difficulty | String | NOT NULL | Poziom trudności sesji determinujący działanie algorytmu przerw (LOW / MEDIUM / HIGH)

createdAt | DateTime | NOT NULL | Znacznik czasu określający dokładną datę i godzinę utworzenia rekordu (domyślnie now()) [cite: 199]

3.3 Diagram bazy danych (tekstowy)

Diagram bazy danych reprezentuje strukturę płaskiej, wydajnej tabeli operacyjnej zorientowanej zadaniowo, która zasila stan frontendu:

Schemat graficzny (uproszczony ASCII):

+-----+

```

|           StudySession           |
+-----+
| [PK] id       : TEXT (UUID, Autogenerated) |
| title        : TEXT (NOT NULL)           |
| duration     : INTEGER (NOT NULL)        |
| difficulty   : TEXT (LOW / MEDIUM / HIGH) |
| createdAt    : NUMERIC (DateTime, Default: NOW) |
+-----+

---

```

4. Schemat nawigacji i przepływu danych

4.1 Opis struktury nawigacji

Aplikacja została zaprojektowana i wdrożona w modelu Single Page Application (SPA), co wyklucza konieczność przeładowywania okna przeglądarki podczas akcji użytkownika[cite: 259, 260]. Przepływ informacji pomiędzy autonomicznymi modułami formularza wejściowego (`StudyForm.tsx`) a panelem prezentacji danych (`StudyList.tsx`) jest koordynowany na poziomie nadrzędnego komponentu stanu `App.tsx` za pomocą mechanizmu podnoszenia stanu (state lifting). Synchronizacja danych z bazą SQLite zachodzi asynchronicznie poprzez zapytania sieciowe HTTP (Fetch API)[cite: 17, 22].

4.2 Mapa przepływu informacji (Data Flow)

App.tsx (Główny Kontener Stanu)

└─ Koordynuje przepływ informacji oraz dystrybucję danych do komponentów podrzędnych

|

└─ Inicjalne pobieranie danych: wywołanie fetch() przy montowaniu widoku

- | └─ HTTP GET —► http://localhost:5000/api/sessions —► Serwer Express
- |
- | └─ Moduł formularza wejściowego: <StudyForm onSessionAdded={fetchSessions} />
- | └─ Obsługa zdarzenia submit -> Walidacja pól -> Przesłanie obiektu JSON
- | └─ HTTP POST —► http://localhost:5000/api/sessions
- |
- | └─ Moduł listy i dashboardu: <StudyList sessions={sessions} />
- | └─ Mapowanie danych na kafelki z wykorzystaniem siatki CSS Grid (RWD)
- | └─ Dynamiczne wstrzyknięcie tekstu wyliczonego powiadomienia (notification) do UI

4.3 Przeptyw głównych akcji użytkownika

Dodanie nowej sesji naukowej

1. Użytkownik uruchamia aplikację w oknie przeglądarki internetowej (na komputerze bądź smartfonie)[cite: 295].
2. Formularz `StudyForm` udostępnia czyste, zwalidowane pola tekstowe i numeryczne[cite: 297].
3. Użytkownik wprowadza temat lekcji, określa czas trwania (np. 45 minut) oraz wybiera poziom trudności `HIGH` [cite: 297].
4. Po kliknięciu przycisku zatwierdzającego, dane są pakowane do formatu JSON i asynchronicznie wysyłane do serwera[cite: 297, 298].
5. Serwer Express odbiera pakiet, Prisma ORM zapisuje rekord w lokalnym pliku bazy SQLite, po czym zwraca status HTTP 201[cite: 298].
6. Frontend reaguje na pomyślny status zapisu, automatycznie czyści pola formularza oraz wywołuje funkcję odświeżającą stan głównego widoku[cite: 299].

Wyliczenie inteligentnych rekomendacji przerw

1. Podczas inicjalizacji lub odświeżenia widoku komponent główny wywołuje asynchroniczne żądanie GET pod adres API[cite: 301].

2. Serwer Express pobiera zestaw surowych rekordów z bazy danych SQLite za pomocą Prisma ORM[cite: 302].
3. Dla każdej pobranej sesji serwer automatycznie uruchamia dedykowany algorytm kalkulatora przerw[cite: 303].
4. Algorytm wylicza bazowy czas (20% z czasu trwania sesji), a następnie modyfikuje go na podstawie pola stopnia trudności (np. dolicza +5 minut bonusu kognitywnego dla poziomu `HIGH`).
5. Serwer generuje gotowy ciąg tekstowy powiadomienia (np. "💡 Zalecana przerwa po sesji: 14 minut.") i dołącza go do obiektu przesyłanego w odpowiedzi JSON[cite: 310].
6. React aktualizuje stan aplikacji, powodując natychmiastowe wyrenderowanie klocków interfejsu z widoczną rekomendacją.

5. Użyte technologie i biblioteki

Biblioteka / Narzędzie | Wersja | Zastosowanie i rola w architekturze systemu

TypeScript | ^5.4+ | Główny język programowania użyty po stronie klienta oraz serwera, zapewniający silne typowanie danych i eliminację błędów [cite: 36, 315]

React.js | ^18.3+ | Reaktywna biblioteka kliencka odpowiedzialna za budowanie komponentów interfejsu SPA i zarządzanie stanem UI [cite: 20, 315]

Express.js | ^4.19+ | Serwerowy framework dla środowiska Node.js odpowiedzialny za obsługę tras HTTP oraz udostępnianie REST API [cite: 15, 17]

Prisma ORM | ^5.12+ | Nowoczesne mapowanie obiektowo-relacyjne zapewniające bezpieczną komunikację z bazą bez pisania zapytań SQL [cite: 16]

SQLite | Latest | Lekka, plikowa relacyjna baza danych niewymagająca instalacji zewnętrznych usług (baza w pliku dev.db) [cite: 16, 315]

Jest | ^29.7+ | Framework testowy używany do tworzenia i automatyzacji testów jednostkowych logiki biznesowej oraz komponentów [cite: 18, 24]

Vite | ^5.2+ | Szybkie, nowoczesne środowisko deweloperskie oraz bundler kodu dla warstwy frontendowej

Cors | ^2.8+ | Middleware dla Express.js umożliwiające bezpieczną komunikację Cross-Origin między różnymi portami [cite: 14]

CSS (Grid / Flexbox) | - | Implementacja responsywnego, nowoczesnego stylowania interfejsu użytkownika (RWD Mobile/Desktop) [cite: 23]

Node.js | ^20+ | Środowisko uruchomieniowe JavaScript po stronie serwerowej, na którym osadzona została aplikacja Express [cite: 15]

Git & GitHub | - | System kontroli wersji oraz zdalne repozytorium służące do wersjonowania kodu i dokumentowania regularności pracy [cite: 28, 30]